

1. What is Docker and Why it Matters in MLOps

Docker is a **containerization platform** that packages an application and all its dependencies into a lightweight, portable **container image**.

It guarantees that *“if it runs on my laptop, it runs on the server.”*

In MLOps, Docker is used to:

- Deploy ML models as microservices.
 - Standardize the training or inference environment.
 - Enable CI/CD pipelines and reproducible builds.
-

2. Docker Architecture (in short)

Docker Client → Docker Daemon (Host) → Docker Images → Containers → Registry

- **Docker Client:** runs CLI commands (`docker build`, `docker run`, etc.)
 - **Docker Daemon (Host):** background service that creates and manages containers.
 - **Images:** blueprints of containers (read-only layers).
 - **Containers:** running instances of images.
 - **Registry:** storage hub for images (e.g., Docker Hub, AWS ECR).
-

3. Core Docker Commands

Task	Command	Meaning
Build image	<code>docker build -t name:tag .</code>	Creates image from Dockerfile
Run container	<code>docker run -p 5000:5000 name:tag</code>	Starts a container and maps port

List images	<code>docker images</code>	Shows available images
List containers	<code>docker ps -a</code>	Shows running/stopped containers
Push to registry	<code>docker push username/name:tag</code>	Uploads image to Docker Hub
Pull from registry	<code>docker pull username/name:tag</code>	Downloads image
Remove container	<code>docker rm id</code>	Deletes container
Stop/start	<code>docker stop id, docker start id</code>	Controls running state

4. Dockerfile — Blueprint for an Image

A **Dockerfile** is a text file with step-by-step build instructions.

◆ Example Dockerfile (CentOS + Python + Bash)

1. Base image

FROM centos:7

2. Install Python and tools

RUN yum update -y && yum install -y python3 python3-pip bash

3. Set work directory inside container

WORKDIR /app

4. Copy local files to container

COPY . /app

5. Install Python dependencies

RUN pip3 install --no-cache-dir -r requirements.txt

6. Define entry command

ENTRYPOINT ["python3", "main.py"]

Line by line

- **FROM**: base image layer (e.g., `ubuntu`, `python:3.10`).
 - **RUN**: executes commands during build (time).
 - **COPY / ADD**: moves files into the image.
 - **WORKDIR**: sets current directory for next commands.
 - **ENTRYPOINT**: default binary to run on `docker run`.
 - **CMD**: default arguments for **ENTRYPOINT**.
 - Each command creates a **new layer** in the image.
-

5. Layers and Caching

Every instruction (**FROM**, **RUN**, **COPY**, etc.) forms a **layer**.

If you rebuild the image and earlier layers haven't changed, Docker reuses them → **faster builds**.

Layers are **read-only**; the container adds a small writable layer on top for runtime changes.

6. ENTRYPOINT vs CMD

Keyword	Purpose	Example
ENTRYPOINT	Defines main binary to run	<code>ENTRYPOINT ["python3", "main.py"]</code>
CMD	Default args to ENTRYPOINT	<code>CMD ["--epochs", "10"]</code>

When combined, **ENTRYPOINT** + **CMD** = binary + arguments.

7. How it fits into MLOps

1. Train model → save `.pkl`.
 2. Write Flask/FastAPI server.
 3. Create Dockerfile and build image.
 4. Push to registry (e.g., Docker Hub).
 5. Deploy via Kubernetes or cloud endpoint.
→ Result: reproducible, scalable model deployment.
-

8. Quick Recap

- **Dockerfile = recipe for containers.**
 - **FROM → RUN → COPY → ENTRYPOINT sequence** creates layers.
 - **Image = read-only blueprint, Container = running instance.**
 - **Push/pull** enables sharing through registries.
 - **ENTRYPOINT / CMD** define container runtime behavior.
-

15 MCQs (with Answers)

① Which command builds a Docker image from a Dockerfile?

A) `docker init` B) `docker build -t name:tag .` C) `docker run build` D) `docker compile`

✓ B

② **FROM** in Dockerfile means:

A) Base image layer B) File to copy C) Working directory D) Entrypoint

✓ A

③ Which Dockerfile instruction executes a command during build time?

A) ENTRYPOINT B) RUN C) CMD D) COPY

✓ B

4 Which command starts a container and maps port 8080 to local 5000?

✓ `docker run -p 5000:8080 image:tag`

5 Difference between Image and Container:

✓ Image = blueprint (read-only); Container = running instance (writable).

6 Which command uploads your built image to Docker Hub?

✓ `docker push username/repo:tag`

7 **ENTRYPOINT** `["python3", "main.py"]` does what?

✓ Defines the default program to execute when container starts.

8 **CMD** is used to:

✓ Provide default arguments for ENTRYPOINT.

9 Which command lists all local Docker images?

✓ `docker images`

10 Docker uses a _____ architecture.

✓ Client-Server (Docker CLI → Daemon)

11 Each Dockerfile instruction creates a new _____

✓ Layer (cached for faster builds)

12 If you change only the last line in Dockerfile, Docker:

✓ Reuses previous layers and rebuilds from the changed layer onwards → fast build.

13 Which command runs a container interactively and opens a shell?

✓ `docker run -it image /bin/bash`

14 To remove a stopped container:

✓ `docker rm container_id`

15 Why is Docker important for MLOps?

✓ Ensures reproducible environments and portable model deployments across cloud and local systems.



10 Short Answers (2–3 sentences each)

1 What is Docker used for in MLOps?

To package models and apps with dependencies into portable containers for consistent deployment across machines.

2 Difference between Docker image and container?

Image is a read-only blueprint; container is a running instance with a writable layer.

3 Purpose of the FROM instruction?

Defines the base image (e.g., `python:3.9`) that provides the starting OS and tools.

4 When is RUN used vs CMD?

`RUN` executes commands during build to create layers; `CMD` runs at container startup.

5 What does ENTRYPOINT do?

Specifies the main executable to run when the container starts.

6 How do Docker layers help with efficiency?

They cache unchanged steps, so re-building after minor edits is much faster.

7 How is Docker different from a VM?

Docker shares the host kernel (very lightweight); VM runs a full OS (heavy).

8 Why set WORKDIR /app in a Dockerfile?

It defines the default working directory so subsequent commands run inside `/app`.

9 What does docker push do?

Uploads your built image to a registry like Docker Hub for others or CI/CD to pull.

10 How does Docker enable CI/CD for ML models?

Each build produces an identical, versioned image that pipelines can test and deploy automatically to production.